Bitácora de Pasantías

Lorena Rojas

The Factory HKA

19 de Enero - 29 de Mayo

Fecha: 19/01/2026

Actividades

- Conocer la interfaz de desarrollo del firmware para el ESP32
- Insvestigar acerca de librerias MQTT para ESP32

Aprendizajes

MQTT es un protocolo de mensajería ligero para IoT con un modelo de publicación/suscripción que ofrece una comunicación fiable en tiempo real con un consumo mínimo de código y ancho de banda. Resulta especialmente beneficioso para dispositivos con recursos limitados y redes con bajo ancho de banda, lo que lo ha convertido en un protocolo ampliamente adoptado en los sectores de IoT, internet móvil, IoV y energía.

El ESP32 , una versión mejorada del ESP8266 , es un microcontrolador de bajo coste y bajo consumo en un chip (SOM). Además del módulo Wi-Fi, el ESP32 también incluye un módulo Bluetooth 4.0. La CPU de doble núcleo opera a una frecuencia de 80 a 240 MHz. Contiene dos módulos Wi-Fi y Bluetooth, y varios pines de entrada y salida. El ESP32 es ideal para proyectos de IoT.

El uso de MQTT en ESP32 ofrece varias ventajas:

- En primer lugar, MQTT es un protocolo de mensajería liviano optimizado para dispositivos y redes restringidos como ESP32 y Wi-Fi, por lo que tiene un impacto mínimo en la energía y el ancho de banda.
- En segundo lugar, MQTT admite diferentes niveles de confiabilidad y calidad de servicio para adaptarse a las capacidades de ESP32. Esta flexibilidad lo hace ideal incluso en redes inestables.
- En tercer lugar, ESP32 y MQTT se utilizan ampliamente en aplicaciones de IoT, lo que permite una buena integración con las soluciones de IoT. El protocolo MQTT también está diseñado para simplificar la integración con plataformas en la nube y permitir el control de dispositivos y la monitorización de datos en redes.

Instalación de VS con MinGW

Logré crear y compilar un archivo programado en C a traves de Visual Studio Code, y sin necesidad de modificar la PATH del sistema. Se logró configurando la extensión de C/C++ desde el VS mediante el siguiente cambio a la PATH:

```
"C:/esp/tools/Espressif/**",
"C:/esp/esp-idf-v5.5.2/**"
```

Dificultades

No se ha podido compilar un proyecto ”get-started”de ESP32 desde el Visual Studio Code, se sigue intentando... El problema esta en que el computador no lee el puerto serial de nuestro ESP32.

Notas

Probar ESP32 en mi computador personal, si tampoco funciona notificar al Ing. Pablo acerca del percanse con el dispositivo.

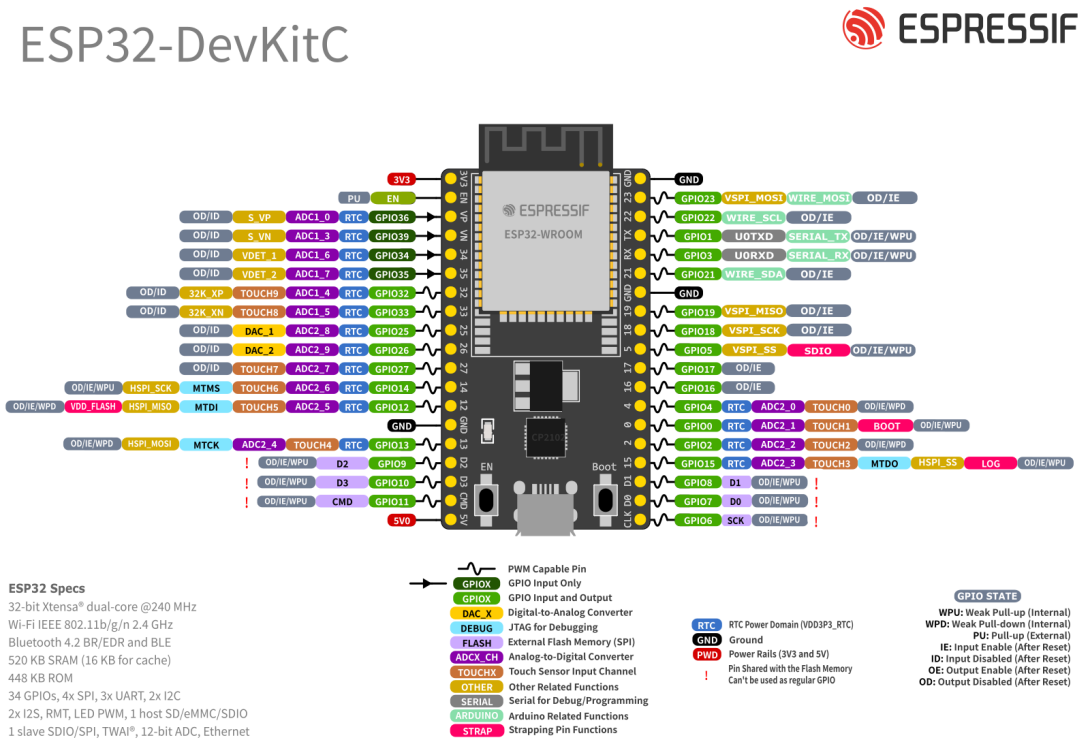
Fecha: 20/01/2026

Actividades

- Realización de mini-proyecto con ESP32 y MQTT
- Posible reunión con Eder y Pablo para definir y/o establecer funcionalidades
- (OPCIONAL) Revisar la interfaz de desarrollo del I-Smart

Aprendizajes

Tarjeta de desarrollo EPS32 DeVKitc V4



Ventajas que ofrece el protocolo MQTT para la escalabilidad y eficiencia en proyectos IoT

El protocolo MQTT (MQ Telemetry Transport) ofrece diversas ventajas diseñadas específicamente para optimizar la escalabilidad y la eficiencia en proyectos de Internet de las Cosas (IoT):

- **Ligereza y bajo consumo de recursos:** Una de las principales fortalezas de este protocolo es su ligereza, lo que lo hace perfecto para ser implementado en dispositivos muy pequeños que cuentan con recursos limitados. Esta eficiencia se traduce en un bajo consumo de recursos del sistema, permitiendo que dispositivos sencillos se comuniquen a través de Internet de forma simple.
- **Capacidad de escalabilidad masiva:** MQTT permite la escalabilidad necesaria para conectar millones de dispositivos a la vez. Esto facilita el manejo de grandes grupos de dispositivos siendo la opción correcta para este proyecto.
- **Arquitectura basada en tópicos y un broker:** El funcionamiento de MQTT se basa en tópicos, que actúan como canales de información a los que los clientes pueden suscribirse o en los que pueden publicar. El broker actúa como el administrador central, encargándose de que la información llegue a su destino y permitiendo que varios dispositivos estén conectados a un mismo tópico simultáneamente,.
- **Niveles de Calidad de Servicio (QoS):** Para asegurar la eficiencia en la entrega de mensajes, el protocolo ofrece tres niveles de servicio:
  - **Nivel 0:** El mensaje se envía una vez sin confirmar recepción.
  - **Nivel 1:** Se garantiza que el mensaje se entregue al menos una vez.

- **Nivel 2:** Se garantiza que el mensaje se entregue exactamente una vez, evitando duplicados.

- **Optimización del tráfico de datos:** En las aplicaciones de IoT, no siempre es necesario enviar datos en cada instante; el uso de intervalos (como cada 5 segundos) ayuda a mantener la eficiencia de la red y del dispositivo. Además, el uso de **brokers avanzados** permite realizar funciones como el filtrado de información y la creación de bases de datos para gestionar mejor el flujo de mensajes.

Conexiones cliente-agente MQTT

MQTT utiliza TCP/IP para conectarse al broker.

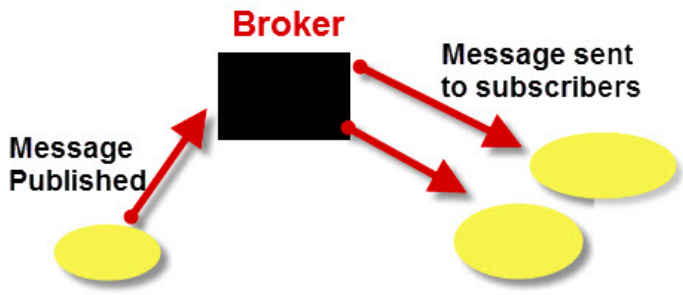
TCP es un protocolo orientado a conexión con corrección de errores y garantiza que los paquetes se reciban en orden.

Se puede considerar que una conexión TCP/IP es similar a una conexión telefónica .

Una vez que se establece una conexión telefónica, puede hablar hasta que una de las partes cuelgue .

La mayoría de los clientes MQTT se conectarán al broker y permanecerán conectados incluso si no están enviando datos.

Las conexiones son reconocidas por el broker mediante un mensaje de reconocimiento de conexión .



MQTT- Publish Subscribe Model

Dificultades

Tuve que realizar una tardada investigacion sobre MQTT para comprender su uso en ESP-IDF. A mitad de la tarde es que pude comenzar con mi proyecto de MQTT.

Mantengo dificultades de compilacion en el Visual Studio Code.

Notas

Se propone el uso de MQTT sobre WebSockets con **Mosquitto**, esto por que es un broker de codigo abierto, gratis y el mas utilizado en MQTT.

Se aumento el repertorio de documentacion en el cuaderno de NotebookLM.

Se inicio un proyecto para probar el ESP32 con MQTT y el Broker Mosquitto, falta trabajar en eso (se buscara concluir mañana).

Fecha: 21/01/2026

Actividades

- Terminar Mini-Proyecto con MQTT y mosquitto
- Poner a prueba la base de datos de mosquitto
- Definir funcionalidades principales a obtener

Aprendizajes

El código de error 201 es: en el ecosistema de Espressif, `WIFI_REASON_NO_AP_FOUND`. Esto significa que el ESP32 no está viendo ninguna red con el nombre "LorenaWiFi". Es un problema de visibilidad.

Los errores de compilación no está realmente en el archivo JSON (ese archivo es solo para que VS Code no marque errores en rojo), sino en el sistema de construcción CMake de ESP-IDF. En ESP-IDF, para mantener el binario pequeño, el compilador no incluye todas las librerías automáticamente. Se tiene que declarar explícitamente al proyecto. Por ejemplo, en mi caso, tuve que editar el archivo `CMakeLists.txt` que está dentro de la carpeta main, tal que:

```
idf_component_register(SRCS "station_example_main.c"
            PRIV_REQUIRES esp_wifi nvs_flash
            INCLUDE_DIRS "."
            REQUIRES esp_http_client esp_event)
```

Se empleo el protocolo de comunicacion HTTP a partir del ESP32, para esto se partio del codigo base suministrado por Espressif en `protocolos/esp_http_client`:

```
esp_http_client_config_t config = {
    .host = CONFIG_EXAMPLE_HTTP_ENDPOINT,
```

```
    .path = "/get",
    .query = "esp",
    .event_handler = _http_event_handler,
    .user_data = local_response_buffer,        // Pass address of local buffe
    .disable_auto_redirect = true,
};
ESP_LOGI(TAG, "HTTP request with url =>");
esp_http_client_handle_t client = esp_http_client_init(&config);

// GET
esp_err_t err = esp_http_client_perform(client);
if (err == ESP_OK) {
    ESP_LOGI(TAG, "HTTP GET Status = %d, content_length = %"PRId64,
            esp_http_client_get_status_code(client),
            esp_http_client_get_content_length(client));
} else {
    ESP_LOGE(TAG, "HTTP GET request failed: %s", esp_err_to_name(err));
}
ESP_LOG_BUFFER_HEX(TAG, local_response_buffer, strlen(local_response_buffer))
```

El cual se adaptó de la siguiente forma, con el fin de enviar datos a una pagina Open Source *ThingsSpeak*:

```
esp_http_client_config_t config = {
    .url = url,
    .method = HTTP_METHOD_GET,
};
esp_http_client_handle_t client = esp_http_client_init(&config);

// GET
esp_err_t err = esp_http_client_perform(client);
if (err == ESP_OK) {
    ESP_LOGI(TAG, "HTTP GET Status = %d, content_length = %"PRId64,
            esp_http_client_get_status_code(client),
            esp_http_client_get_content_length(client));
} else {
    ESP_LOGE(TAG, "HTTP GET request failed: %s", esp_err_to_name(err));
}

esp_http_client_cleanup(client);
```

Con estos cambios se logro satisfactoriamente la visualizacion de datos en la web, sentando las bases para avanzar y transformar el proyecto a un protocolo MQTT mediante `mosquitto`.

Con mosquitto, se pudo crear un servidor local MQTT teniendo como host el computador personal, siendo el codigo base el siguiente:

```
static void mqtt_event_handler(void *handler_args, esp_event_base_t base, int3
{
    ESP_LOGD(TAG, "Event dispatched from event loop base=%s, event_id=%" PRIi3
    esp_mqtt_event_handle_t event = event_data;
    esp_mqtt_client_handle_t client = event->client;
    int msg_id;
    switch ((esp_mqtt_event_id_t)event_id) {
    case MQTT_EVENT_CONNECTED:
        ESP_LOGI(TAG, "MQTT_EVENT_CONNECTED");
        msg_id = esp_mqtt_client_subscribe(client, "/topic/qos0", 0);
        ESP_LOGI(TAG, "sent subscribe successful, msg_id=%d", msg_id);

        msg_id = esp_mqtt_client_subscribe(client, "/topic/qos1", 1);
        ESP_LOGI(TAG, "sent subscribe successful, msg_id=%d", msg_id);

        msg_id = esp_mqtt_client_unsubscribe(client, "/topic/qos1");
        ESP_LOGI(TAG, "sent unsubscribe successful, msg_id=%d", msg_id);
        break;
    case MQTT_EVENT_DISCONNECTED:
        ESP_LOGI(TAG, "MQTT_EVENT_DISCONNECTED");
        break;

    case MQTT_EVENT_SUBSCRIBED:
        ESP_LOGI(TAG, "MQTT_EVENT_SUBSCRIBED, msg_id=%d", event->msg_id);
        msg_id = esp_mqtt_client_publish(client, "/topic/qos0", "data", 0, 0,
        ESP_LOGI(TAG, "sent publish successful, msg_id=%d", msg_id);
        break;
    case MQTT_EVENT_UNSUBSCRIBED:
        ESP_LOGI(TAG, "MQTT_EVENT_UNSUBSCRIBED, msg_id=%d", event->msg_id);
        break;
    case MQTT_EVENT_PUBLISHED:
        ESP_LOGI(TAG, "MQTT_EVENT_PUBLISHED, msg_id=%d", event->msg_id);
        break;
    case MQTT_EVENT_DATA:
        ESP_LOGI(TAG, "MQTT_EVENT_DATA");
        printf("TOPIC=%.*s\r\n", event->topic_len, event->topic);
        printf("DATA=%.*s\r\n", event->data_len, event->data);
        if (strncmp(event->data, "send binary please", event->data_len) == 0)
            ESP_LOGI(TAG, "Sending the binary");
            send_binary(client);
        }
        break;
    case MQTT_EVENT_ERROR:
        ESP_LOGI(TAG, "MQTT_EVENT_ERROR");
        if (event->error_handle->error_type == MQTT_ERROR_TYPE_TCP_TRANSPORT)
```

```
            ESP_LOGI(TAG, "Last error code reported from esp-tls: 0x%x", event
            ESP_LOGI(TAG, "Last tls stack error number: 0x%x", event->error_ha
            ESP_LOGI(TAG, "Last captured errno : %d (%s)",  event->error_handl
                     strerror(event->error_handle->esp_transport_sock_errno))
        } else if (event->error_handle->error_type == MQTT_ERROR_TYPE_CONNECTI
            ESP_LOGI(TAG, "Connection refused error: 0x%x", event->error_handl
        } else {
            ESP_LOGW(TAG, "Unknown error type: 0x%x", event->error_handle->err
        }
        break;
    default:
        ESP_LOGI(TAG, "Other event id:%d", event->event_id);
        break;
    }
}

static void mqtt_app_start(void)
{
    const esp_mqtt_client_config_t mqtt_cfg = {
        .broker = {
            .address.uri = CONFIG_BROKER_URI,
            .verification.certificate = (const char *)mqtt_eclipseprojects_io_
        },
    };

    ESP_LOGI(TAG, "[APP] Free memory: %" PRIu32 " bytes",
            esp_get_free_heap_size());
    esp_mqtt_client_handle_t client = esp_mqtt_client_init(&mqtt_cfg);
    /* The last argument may be used to pass data to the event handler, in thi
    esp_mqtt_client_register_event(client, ESP_EVENT_ANY_ID,
                                   mqtt_event_handler, NULL);
    esp_mqtt_client_start(client);
}
```

El cual se modifico de la siguiente forma:

```
static void mqtt_event_handler(void *handler_args,
esp_event_base_t base, int32_t event_id, void *event_data)
{
    ESP_LOGD(TAG, "Event dispatched from event loop base=%s,
            event_id=%" PRIi32, base, event_id);
    esp_mqtt_event_handle_t event = event_data;
    esp_mqtt_client_handle_t client = event->client;
    switch ((esp_mqtt_event_id_t)event_id) {
    case MQTT_EVENT_CONNECTED:
        ESP_LOGI(TAG, "MQTT_EVENT_CONNECTED");
        //msg_id = esp_mqtt_client_subscribe(client, "pruebahka/lorena1", 0);
```

```
        //ESP_LOGI(TAG, "sent subscribe successful, msg_id=%d", msg_id);

        xTaskCreate(mqtt_publish_task, "mqtt_publish_task", 4096,
                    NULL, 5, NULL);
        break;

    case MQTT_EVENT_DISCONNECTED:
        ESP_LOGI(TAG, "MQTT_EVENT_DISCONNECTED");
        break;

    case MQTT_EVENT_SUBSCRIBED:
        ESP_LOGI(TAG, "MQTT_EVENT_SUBSCRIBED, msg_id=%d", event->msg_id);
        break;

    case MQTT_EVENT_UNSUBSCRIBED:
        ESP_LOGI(TAG, "MQTT_EVENT_UNSUBSCRIBED, msg_id=%d", event->msg_id);
        break;

    case MQTT_EVENT_PUBLISHED:
        ESP_LOGI(TAG, "MQTT_EVENT_PUBLISHED, msg_id=%d", event->msg_id);
        break;

    case MQTT_EVENT_DATA:
        ESP_LOGI(TAG, "MQTT_EVENT_DATA");
        printf("TOPIC=%.*s\r\n", event->topic_len, event->topic);
        printf("DATA=%.*s\r\n", event->data_len, event->data);
        break;

    case MQTT_EVENT_ERROR:
        ESP_LOGI(TAG, "MQTT_EVENT_ERROR");
        if (event->error_handle->error_type ==
            MQTT_ERROR_TYPE_TCP_TRANSPORT) {
            ESP_LOGI(TAG, "Last error code reported from esp-tls: 0x%x",
                    event->error_handle->esp_tls_last_esp_err);
            ESP_LOGI(TAG, "Last tls stack error number: 0x%x",
                    event->error_handle->esp_tls_stack_err);
            ESP_LOGI(TAG, "Last captured errno : %d (%s)",
                    event->error_handle->esp_transport_sock_errno,
                     strerror(event->error_handle->esp_transport_sock_errno))
        } else if (event->error_handle->error_type ==
                    MQTT_ERROR_TYPE_CONNECTION_REFUSED) {
            ESP_LOGI(TAG, "Connection refused error: 0x%x",
                    event->error_handle->connect_return_code);
        } else {
            ESP_LOGW(TAG, "Unknown error type: 0x%x",
                    event->error_handle->error_type);
        }
        break;
```

```
        default:
            ESP_LOGI(TAG, "Other event id:%d", event->event_id);
            break;
        }
    }

    static void mqtt_app_start(void)
    {
        // Es mejor inicializar la estructura a cero primero
        esp_mqtt_client_config_t mqtt_cfg = { 0 };

        mqtt_cfg.broker.address.hostname = "10.142.223.70";
        mqtt_cfg.broker.address.port = 1883;
        mqtt_cfg.broker.address.transport = MQTT_TRANSPORT_OVER_TCP;

        mqtt_cfg.broker.verification.use_global_ca_store = false;

        ESP_LOGI(TAG, "Iniciando MQTT hacia el Host: %s",
                mqtt_cfg.broker.address.hostname);

        esp_mqtt_client_handle_t client = esp_mqtt_client_init(&mqtt_cfg);
        mqttClient = client;

        if (client == NULL) {
            ESP_LOGE(TAG, "Error: No se pudo inicializar el cliente
                    puntero nulo)");
            return;
        }

        /* Registrar eventos */
        esp_mqtt_client_register_event(client, ESP_EVENT_ANY_ID,
                                       mqtt_event_handler, NULL);

        /* Iniciar el cliente */
        esp_err_t err = esp_mqtt_client_start(client);
        if (err != ESP_OK) {
            ESP_LOGE(TAG, "Error al iniciar el cliente MQTT: %s",
                    esp_err_to_name(err));
        }
    }
```

Luego se elavoro una version que envia numeros random desde el ESP32 hacia el broker y de este modo el computador lo recibe conectado al mismo broker. Ya en el computador se elaboro un Scrip en Python que se conecta al mismo broker y recibe la información, de modo que la guarda en un *dataframe* y se genera un archivo CSV que contiene dicha informacion,.siendo este el primer paso para generar una base de datos, que en un futuro

se pasara a SQLite.

	A
1	Valor
2	Valor: 13
3	Valor: 94
4	Valor: 33
5	Valor: 63
6	Valor: 64
7	Valor: 16
8	Valor: 66
9	Valor: 73
10	Valor: 10
11	Valor: 66

Dificultades

En vista que el broker con Mosquitto funciona desde un establecimiento puesto localmente, el resultado final se veria afectado por las limitaciones del hardware en cuestion. Por lo tanto, evidentemente se trabajara con un prototipado que maneje solo 100 usuarios mediante el broker de HiveMQ en la nube.

Notas

Se continuó con el tema de escoger las funcionalidades del proyecto final, para ello se busca responder las siguientes preguntas:

- ¿Qué parámetros específicos de los ”datos operativos”son críticos para el monitoreo?

  - Estado

  - Errores

  - Actividad de impresión

  -

- ¿Que sensores específicos tiene el equipo fiscal?

- En caso de tenerlos, ¿A través de qué sensores específicos del equipo fiscal se realizará la captura?

- ¿Cómo debe actuar el firmware del ESP32 ante un intento fallido de publicación por MQTT?

- ¿Qué mecanismos de diagnóstico y trazabilidad se requieren para registrar el historial de eventos del equipo?
- ¿Qué alertas específicas debe mostrar la interfaz web o móvil en tiempo real ante un error del equipo fiscal?
- ¿Es necesario que el dashboard permita enviar comandos de vuelta al equipo (comunicación bidireccional)?
- ¿Cómo se deben gestionar y estructurar los datos dentro del servidor MQTT para su posterior consulta?

Entre otras, se destaca que es lo primordial para una primera version del sistema,

1. Login: Esencial para cumplir con la çomunicación segura".
2. User Management (Básico): Solo la capacidad de gestionar tu propio usuario y quizás un rol de administrador para las pruebas iniciales de validación.
3. Device (Add a device): Se necesitara una interfaz para registrar los microcontroladores ESP32 que se conectarán al sistema.
4. Log capture: Esto se vincula directamente con los "mecanismos de trazabilidadz la recolección de datos operativos"(estado, errores, actividad).
5. System params: Útil para configurar los parámetros de MQTT que el firmware debe utilizar.
6. Dashboard: Una versión simplificada que muestre los datos operativos capturados desde el equipo fiscal.
7. Audit logs: Fundamental para la "validación de robustez ante fallos", permitiéndote ver qué pasó si la conexión se perdió.
8. Firmware Management (Básico): No necesariamente una actualización remota (OTA) compleja aún, sino asegurar que el firmware embebido desarrollado pueda comunicarse con el servidor MQTT.
9. TLS Certificate: Si buscas una çomunicación segura"desde la primera versión, la gestión de certificados para MQTT es prioritaria.

Fecha: 22/01/2026

(Jueves)

Actividades

- Descargar Altium
- Conseguir interfaz de desarrollo del iSmart
- Estudiar interfaz de desarrollo del iSmart

Aprendizajes

Por la razón de ser pasante no será posible tener disposicion de la interfaz iSmart para desarrollo, por lo tanto se seguirá trabajando con el ESP32, buscando similar el estado S1 de una impresora fiscal, por lo menos hasta que se desarrolle el prototipo con el que voy a trabajar.

En vista de eso, se elaboró la propuesta de otro objetivo para el día: Çonectar ESP32 y Scrip de Python a HiveMQ con MQTTv5". Siendo este un desafio porque presentaba muchos mas protocolos de seguridad, pero cumplia mucho mejor los criterios de la prupuesta donde se especifica que debe ser un sistema seguro.

Dificultades

Notas

Para la siguiente semana se buscará diseñar la arquitectura completa de lo que seria el sistema IoT, mientras que en el transcurso del fin de semana se estudiara la documentacion de la impresora fiscal con el fin de trabajar de manera fluida la semana que viene.

Fecha: 26/01/2026

Actividades

- Leer documentación referente a los protocolos de comunicación en los equipos fiscales
- Diseñar arquitectura del sistema IoT
- Diseñar arquitectura de comunicaciones MQTT

Aprendizajes

Comunicación entre el PC y la impresora fiscal

El protocolo de comunicación de las impresoras fiscales se basa en el estándar RS232 de comunicación serial. Para esto, es necesaria una interfaz de aplicación que gestione este protocolo, esto es, que sea capaz de enviar los comandos desde el computador hacia la impresora e interpretar las respuestas que está retorna. Estos comandos corresponden a protocolos seriales almacenados en el firmware de la impresora.

Los comandos de estos protocolos pueden ser enviados a la impresora de dos maneras: directamente a través del manejo del puerto serial (llamado Protocolo Directo), o utilizando interfaces de programación de aplicaciones (API, Application Programming Interface) las cuales dependen del sistema operativo a utilizar y del lenguaje de programación utilizado para desarrollar el Sistema Administrativo que estará asociado la impresora.

Los parámetros de configuración del puerto serial son los siguientes:

Transmisión de data	Serial, Asíncrona
Baud rate	9600 bps
Bit de datos	8 bits
Paridad	Par
Bits de stop	1

Estructura de la trama de comunicación

La trama de comunicación es el conjunto de datos que debe enviarse a la impresora para que cumpla determinada instrucción; debe enviarse en orden y está constituida siempre por cuatro secciones.

Las secciones de la trama de comunicación son las siguientes:

- Carácter de inicio de trama (STX): representado por el carácter 0x02h, es un valor reservado únicamente a este fin.
- DATA: Es el comando y sus argumentos, enviados a la impresora para que ejecute una determinada acción.
- Carácter de Fin de Trama (ETX): representado por el carácter 0x03h indica el fin de la trama y es un valor reservado únicamente a este fin.
- LRC: Su valor es el OR exclusivo (XOR) entre la DATA y ETX, dirigido a la detección de error de la trama.

ASCII	STX	I	0	X	ETX	LRC
HEX	02	**49**	**30**	**58**	03	22
	Inicio de trama	DATA			Fin de Trama	LRC

Tabla 4. Ejemplo de instrucción con protocolo directo.

Leer estado

Para determinar el estado en que se encuentra la impresora fiscal, se envía un Enquirement (ENQ=0x05h)

Este comando se envía a la impresora fiscal para determinar el estado en que se encuentra y si existe un error, evaluarlo. Cuando se envía un ENQ a la impresora, esta responde con una trama similar a la de recepción, donde DATA es un par de bytes que contienen la información del Estado y el posible Error de la impresora.

La impresora responderá una trama con la siguiente estructura:

Dónde:

- STS1 corresponde al Estado de la impresora.
- STS2 corresponde al Error de la impresora.

Las siguientes tablas contienen los valores frecuentes para los bytes de Status (STS1) y Error (STS2) de las impresoras fiscales:

Valores de Status (STS1)	
0x40	Modo Entrenamiento y en Espera
0x41	Modo Entrenamiento y en medio de una Transacción Fiscal
0x42	Modo Entrenamiento y en medio de una Transacción No fiscal
0x60	Modo Fiscal y en Espera
0x68	Modo Fiscal con la MF llena y en Espera
0x61	Modo Fiscal y en medio de una Transacción Fiscal
0x69	Modo Fiscal con la MF llena y en medio de una Transacción Fiscal
0x62	Modo Fiscal y en medio de una Transacción No fiscal
0x6A	Modo Fiscal con la MF llena y en Transacción No fiscal

Valores de Error (STS2)	
0x40	Ningún error
0x48	Error gaveta
0x41	Error sin papel
0x42	Error mecánico de la impresora / papel
0x43	Error mecánico de la impresora y fin de papel
0x60	Error fiscal
0x64	Error en la memoria fiscal
0x6C	Error memoria fiscal llena

Leer status de información

Se puede tener acceso a la información que posee la impresora Fiscal, dicha información es repartida en diversos status informativos.

En el caso de protocolo directo, debe enviar la trama de la solicitus que desee y leer la respuesta en el puerto de comunicaciones basándose en las tablas de respuesta aquí

discritas.

STATUS S1 Este comando permite consultar información referente a parámetros de la impresora fiscal como Serial de la misma, RIF, datos de factura, entre otros. Este comando es posible ejecutarlo en cualquier condición.

Descripción	PP9-PLUS, TD1140				
	Librería		Protocolo Directo		L
	Desde	Hasta	Desde	Hasta	
Status y Número de Cajero.	0	3	0	3	4
Subtotal de Ventas (Bs.)	5	21	4	20	17
Número de la Última Factura.	23	30	21	28	8
Cantidad de Facturas Emitidas en el día.	32	36	29	33	5
Número de la última nota de débito.	38	45	34	41	8
Cantidad de notas de débito del día.	47	51	42	46	5
Número de Última Nota de Crédito.	53	60	47	54	8
Cantidad de Notas de Crédito.	62	66	55	59	5
Número del Último Documento No Fiscal.	68	75	60	67	8
Cantidad de Documentos No Fiscales.	77	81	68	72	5
Contador de Cierres Diarios (Z).	83	86	73	76	4
Contador de Reportes de Memoria Fiscal.	88	91	77	80	4
RIF.	93	103	81	91	11
Número de Registro de la Máquina.	105	114	92	101	10
Hora Actual de la Impresora.	116	121	102	107	6
Fecha Actual de la Impresora.	123	128	108	113	6
Longitud Total.	**129**		**113**		**129**

Dificultades

Notas

Durante el transcurso de la semana se buscara cumplir con los objetivos de las semanas 3 y 4 del plan de trabajo, que se centran en el **Diseño de la arquitectura IoT** y el **Diseño de la arquitectura de comunicaciones MQTT**, se debe definir los planos técnicos y lógicos que permitirán que el ESP32 actúe como el puente entre el equipo fiscal y la nube.

A continuación, se detalla qué debería abarcar exactamente cada actividad:

1. **Diseño de la arquitectura IoT:** Esta actividad debe definir la estructura macro del sistema, integrando el hardware, los protocolos y la plataforma de visualización.
   - **Diagrama de bloques del sistema:** Debes representar el flujo de datos desde el equipo fiscal hacia el ESP32 (vía serial RS232), y desde este hacia el servidor MQTT y el dashboard final.
   - **Interfaz física (Capa de Dispositivo):** Especificar la conexión serial con la impresora utilizando el estándar RS232 con los parámetros técnicos obligatorios: 9600 bps, 8 bits de datos, Paridad Par y 1 bit de parada.
   - **Capa de Red:** Definir el uso de la red Wi-Fi como medio de transporte para los paquetes MQTT.
   - **Definición de datos operativos:** Determinar exactamente qué campos del manual (como el Status S1 o S2) se van a recolectar (Serial, RIF, actividad de impresión, errores) para ser transmitidos.
2. **Diseño de la arquitectura de comunicaciones MQTT:** Aquí se debe configurar la lógica del protocolo para garantizar una comunicación segura y confiable, aprovechando que el ESP-MQTT ya soporta MQTT v5.0.
   - **Estructura de Tópicos (Topic Tree):** Diseñar una jerarquía de tópicos eficiente y descriptiva (ej. `empresa/dispositivo01/estado`), evitando el uso de espacios o barras iniciales innecesarias.
   - **Niveles de Calidad de Servicio (QoS):** Decidir qué nivel de QoS usarás. Para datos críticos fiscales, se recomienda QoS 1 (al menos una vez) o QoS 2 (exactamente una vez) para evitar pérdida de información.
   - **Gestión de Sesiones y Mensajes:**
     - **Sesiones Persistentes:** Definir si el broker debe almacenar mensajes mientras el ESP32 esté offline.
     - **Last Will and Testament (LWT):** Configurar un ”testamento”para que el servidor notifique automáticamente al dashboard si el equipo fiscal se desconecta de forma inesperada.
   - **Seguridad:** Definir los mecanismos de autenticación (usuario y contraseña) y el uso de TLS/SSL para el cifrado de la comunicación con el servidor remoto.

- **Propiedades de MQTT v5.0:** Evaluar el uso de Topic Aliases para reducir el tamaño de los paquetes y User Properties para enviar metadatos adicionales sin afectar el payload.

Al finalizar esta semana, se debería contar con diagramas de flujo y una tabla de especificaciones que dicten cómo se programará el firmware y se configurará el servidor en las etapas posteriores del cronograma.

Diagrama de arquitectura IoT

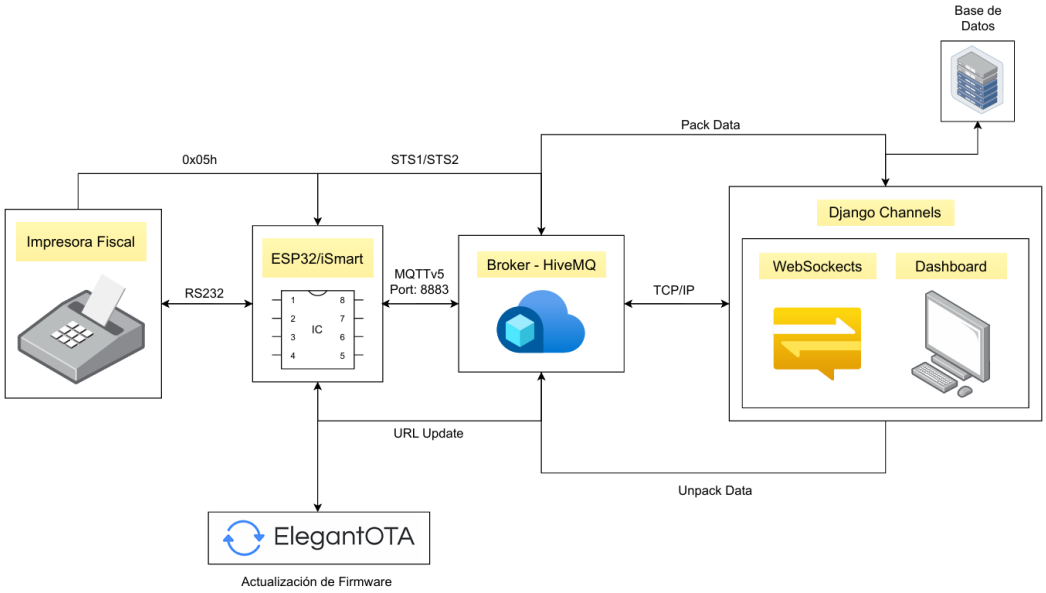


Figura 1: Arquitectura diseñada para el sistema IoT.

Diagrama de arquitectura de comunicación MQTT

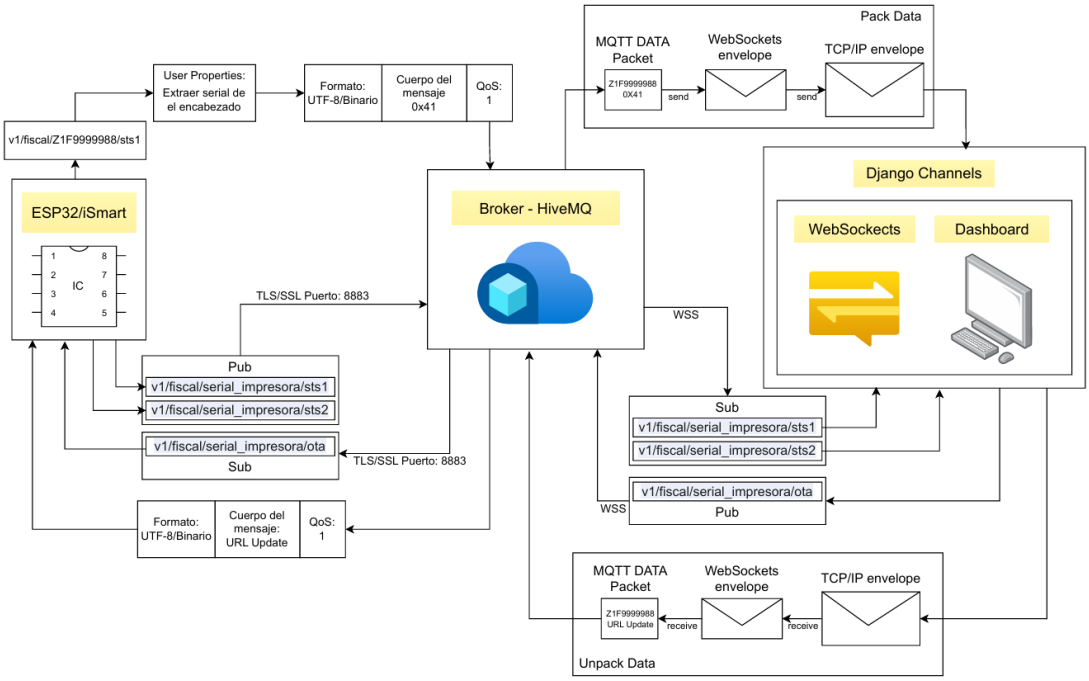


Figura 2: Arquitectura diseñada para la comunicación MQTT.

Fecha: 27/01/2026

Actividades

- Comenzar codigo para leer y procesar los datos en el ESP32

Aprendizajes

Conceptos básicos de una Tarea (Task)

Una tarea es simplemente una función de C que contiene un bucle infinito (while(1)) y que nunca debe retornar. En nuestro caso, necesitamos al menos una tarea que se encargue de ïnterrogar.ª la impresora periódicamente.

La función principal para crear tareas es:

- `xTaskCreate()`: Crea una tarea y deja que el sistema decida en qué núcleo ejecutarla.
- `xTaskCreatePinnedToCore()`: Muy recomendada en el ESP32 para asegurar que una tarea (como la de la impresora) no interfiera con las tareas críticas de red (como Wi-Fi/MQTT).

Ejemplo práctico: Tarea de Interrogación (Polling) Basado en el flujo de diseño donde el ESP32 debe enviar un `0x05h` (ENQ) para solicitar el estado, así se vería la estructura de una tarea simple:

```
// Esta es la función de la tarea
void printer_polling_task(void *pvParameters) {
    uint8_t enq_cmd = 0x05; // Carácter ENQ según el manual

    while(1) {
        // 1. Enviar el comando por UART
        uart_write_bytes(UART_NUM_1, (const char*) &enq_cmd, 1);

        // 2. Esperar un tiempo antes de volver a preguntar
        // Es vital usar vTaskDelay para no bloquear el procesador
        vTaskDelay(pdMS_TO_TICKS(5000)); // Pregunta cada 5 segundos
    }
}

// Así la creas en tu app_main()
void app_main(void) {
    // ... Configuración de UART que ya hiciste ...

    // Crear la tarea: (función, nombre, stack, parámetros, prioridad, handle
    xTaskCreate(printer_polling_task, "printer_task", 4096, NULL, 5, NULL);
}
```

Los pines UART en nuestro ESP32 son:

- UART0: TX (GPIO1), RX (GPIO3)
- UART1: TX (GPIO10), RX (GPIO9). IMPORTANTE: En el DevKitC V4, estos pines suelen estar conectados a la memoria Flash interna. Si se intenta usarlos, el ESP32 se reiniciará constantemente o no encenderá. No usar. Si se necesita usar, se pueden mover el UART1 hacia otros pines libre, por ejemplo: GPIO18 (TX) y GPIO19 (RX).
- UART2: TX (GPIO17), RX (GPIO16)

Dificultades

Para simular una impresora fiscal desde mi computador necesito conectar otro puerto serial (UART2) en mi ESP32.

Notas

Tratando de adaptar un cable RJ12, para comunicar el computador y el ESP32, por instrucción de Antonio aprendi a soldar cables.

Fecha: 28/01/2026

Actividades

- Lograr doble puerto serial en el computador
- Probar código hecho el día anterior
- Seguir avanzando en la version 1 del firmware ESP32

Aprendizajes

Logrando probar el codigo hecho el dia anterior probamos diferentes resultados, desde tramas de bits erroneas

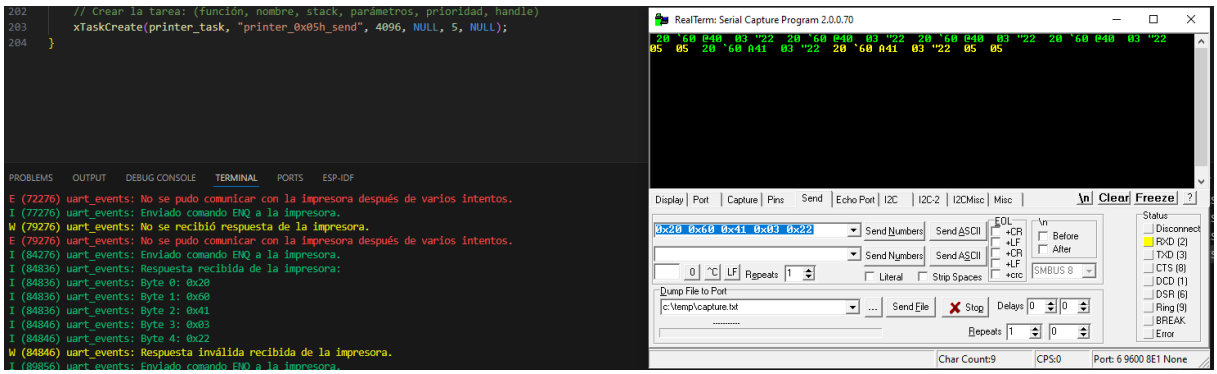


Figura 3: Respuesta obtenida al enviar la trama: 0x20 0x60 0x41 0x03 0x22.

Hasta tramas de bits correctas y publicando el estado de la impresora por puerto serial:

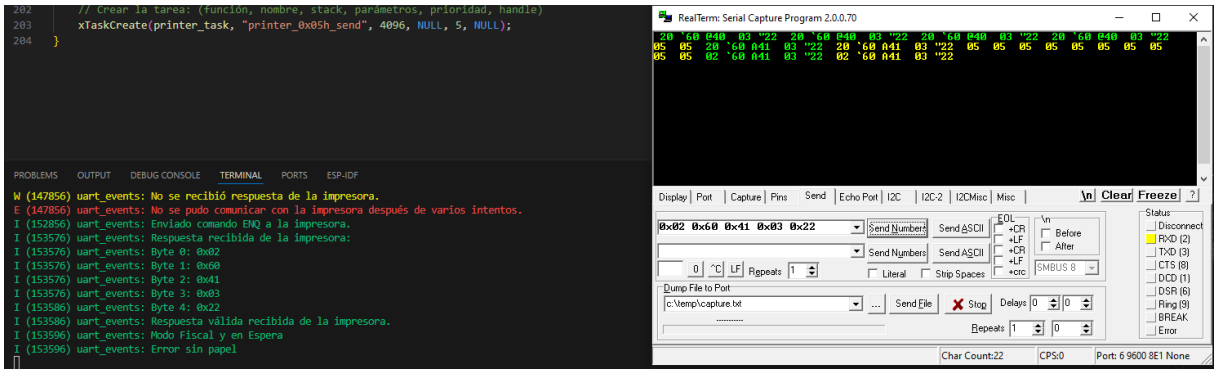


Figura 4: Respuesta obtenida al enviar la trama: 0x02 0x60 0x41 0x03 0x22.

Dificultades

Se tuvieron dificultades en el cable prolofic por temas de drivers, pero con instrucción de Elio se pudo solventar.

En el caso del segundo monitor, Realterm, no es posible enviar la una cadena de bits tan larga como lo es STATUS S1, se barajea la opción de cambiar este monitor si no se resuelve el problema.

Notas

Continuamos el codigo buscando procesar los datos de STATUS S1, donde por precaución se extraeran todos, en un futuro de no necesitarlos se eliminara del código.

Fecha: 30/01/2026

Actividades

- Culminar la captura y procesamiento de datos para SATATUS S1
- Mejorar eficiencia y robustez del código

Aprendizajes

Dificultades

Mandar la trama de datos de la impresora a traves del Realterm se volvio un proceso complicado e ineficiente, ya que este tenia combinaciones de caracteres (hexadecimal y

ASCII) por lo que se recurrio al uso de un Scrip en python que automaticamente simulaba el comportamiento de una impresora fiscal, mediante el siguiente codigo:

```
import serial
import time
import sys

def simulador_depuracion():
    """Simulador con depuración detallada"""
    try:
        ser = serial.Serial(
            port='COM6',
            baudrate=9600,
            bytesize=serial.EIGHTBITS,
            parity=serial.PARITY_EVEN,
            stopbits=serial.STOPBITS_ONE,
            timeout=1
        )

        print("=" * 60)
        print("SIMULADOR DE IMPRESORA FISCAL - MODO DEPURACIÓN")
        print("=" * 60)
        print(f"Puerto: {ser.port}")
        print(f"Baudrate: {ser.baudrate}")
        print(f"Bytesize: {ser.bytesize}")
        print(f"Parity: {ser.parity}")
        print(f"Stopbits: {ser.stopbits}")
        print("=" * 60)
        print("Esperando comandos...")
        print("Presiona Ctrl+C para salir")
        print("=" * 60)

        contador = 0

        while True:
            # Leer datos byte por byte
            if ser.in_waiting > 0:
                byte = ser.read(1)
                contador += 1

                print(f"\n[{contador}] Byte recibido:")
                print(f"  Hexadecimal: 0x{byte.hex().upper()}")
                print(f"  Decimal: {int.from_bytes(byte, 'big')}")
                print(f"  Binario: {bin(int.from_bytes(byte, 'big'))[2:].zfill(8)}")

                # Intentar interpretar como ASCII
                try:
                    ascii_char = byte.decode('ascii')
```

```
        print(f"  ASCII: '{ascii_char}'")
    except:
        print(f"  ASCII: No es ASCII válido")

    # Mostrar tipo de comando
    if byte == b'\x05':
        print(f"  TIPO: ENQ (Consulta de estado)")
    elif byte == b'\x4C':
        print(f"  TIPO: STATUS S1")
    elif byte == b'\x02':
        print(f"  TIPO: STX (Inicio de trama)")
    elif byte == b'\x03':
        print(f"  TIPO: ETX (Fin de trama)")
    elif byte == b'\x15':
        print(f"  TIPO: NAK (No reconocimiento)")
    elif byte == b'\x06':
        print(f"  TIPO: ACK (Reconocimiento)")
    else:
        print(f"  TIPO: Desconocido")

    # Responder según el byte recibido
    if byte == b'\x05':  # ENQ
        print("  -> Enviando respuesta ENQ: 0x02 0x60 0x40 0x03")
        respuesta = b'\x02\x60\x40\x03'
        ser.write(respuesta)
        ser.flush()

    elif byte == b'\x4C':  # STATUS S1
        print("  -> Enviando respuesta STATUS S1")
        # Construir datos de prueba
        datos = "0010000000000000000000000000000000000000000000
        00000000000000000000000000000000000000J3121711970
        Z1F9999988123000300126"

        # Enviar STX + datos + ETX
        respuesta = b'\x02' + datos.encode('ascii') + b'\x03'
        print(f"  -> Longitud respuesta: {len(respuesta)} bytes")
        print(f"  -> STX: 0x02, ETX: 0x03")
        print(f"  -> Datos ASCII: {len(datos)} caracteres")

        ser.write(respuesta)
        ser.flush()

    else:
        print(f"  -> Byte desconocido, NO respondiendo")
        # NO enviar NAK para evitar el bucle infinito
        # ser.write(b'\x15')
```

```
                print("-" * 40)

            time.sleep(0.01)

    except KeyboardInterrupt:
        print("\n\nSimulador detenido por usuario")
    except Exception as e:
        print(f"\nError: {e}")
    finally:
        if 'ser' in locals() and ser.is_open:
            ser.close()
            print("Puerto serial cerrado")

if __name__ == "__main__":
    simulador_depuracion()
```

Notas

Se logró mandar e interpretar el comando STATUS S1 en nuestra primera versión de firmware para ESP32 obteniendo este primer resultado preliminar que luego mandaremos a traves de protocolo MQTT:

```
I (13297) uart_events: === STATUS S1 ===
I (13297) uart_events: Número de Cajero: [0010]
I (13307) uart_events: Subtotal Ventas: [000000000000000000]
I (13307) uart_events: Última Factura: [00000000]
I (13317) uart_events: Facturas hoy: [00000]
I (13317) uart_events: RIF: [J3121711970]
I (13317) uart_events: Número Registro: [Z1F9999988]
I (13317) uart_events: RIF: [J3121711970]
I (13317) uart_events: Número Registro: [Z1F9999988]
I (13317) uart_events: Número Registro: [Z1F9999988]
I (13327) uart_events: Hora: [123000]
I (13327) uart_events: Fecha: [300126]
```

Es evidente que hay redundancias y repeticiones, por lo cual se trabajara en solventar, pero tambien vemos que la data esta organizada correctamente y los datos coinciden con sus respectivos campos.

Se logró conseguir una version final que funciona correctamente:

```
I (12278) uart_events: Enviado comando ENQ a la impresora.
I (12298) uart_events: Respuesta válida recibida de la impresora.
I (12298) uart_events: Modo Fiscal y en Espera
```

```
I (12298) uart_events: Ningún error
I (12398) uart_events: Enviado carácter STATUS S1 (0x4C) a la impresora.
I (17898) uart_events: Procesando STATUS S1, len=116
I (17898) uart_events: STX encontrado en posición: 0
I (17898) uart_events: === STATUS S1 ===
I (17898) uart_events: Número de Cajero: [0010]
I (17898) uart_events: Subtotal Ventas: [00000000000000000]
I (17908) uart_events: Número de Última Factura: [00000000]
I (17908) uart_events: Cantidad de Facturas Emitidas: [00000]
I (17918) uart_events: Número de Última Nota de Débito: [00000000]
I (17918) uart_events: Cantidad de Notas de Débito del Día: [00000]
I (17928) uart_events: Número de Última Nota de Crédito: [00000000]
I (17938) uart_events: Cantidad de Notas de Crédito: [00000]
I (17938) uart_events: Número del Último Documento No Fiscal: [00000000]
I (17948) uart_events: Cantidad de Documentos No Fiscales: [00000]
I (17948) uart_events: Contador de Cierres Diarios (Z): [0000]
I (17958) uart_events: Contador de Reportes de Memoria Fiscal: [0000]
I (17968) uart_events: RIF: [J3121711970]
I (17968) uart_events: Número Registro de la Máquina: [Z1F9999988]
I (17978) uart_events: Hora Actual de la Impresora: [123000]
I (17978) uart_events: Fecha Actual de la Impresora: [300126]
```

Falto por mejorar la eficiencia y robustez del codigo, una vez logrado eso se avanzará con la siguiente fase del proyecto Ïmplementación de comunicación Wi-Fi y MQTT en el ESP32”

Fecha: 02/02/2026

Actividades

- Parametrizar variables de información que se enviaran a través de MQTT
- Mejorar eficiencia de código firmware V1
- (OPCIONAL) Comenzar adaptación a WiFi y MQTT, comenzando Firmware V2

Aprendizajes

Mejorando la estructura del codigo, buscaré implementar las siguientes mejoras:

- Sistema de reconexión y timeouts.
- Seguridad en acceso de memoria.

- Configuración centralizada (archivo header).
- Mejorar manejo de errores.
- Optimización del performance, volviendolo mas personalizado hacia su objetivo.

Se implementó la función `xTaskGetTickCount()` donde devuelve el conteo total de ”ticks”(ciclos de reloj del sistema) que han transcurrido desde que el planificador de tareas (scheduler) se inició tras el arranque del ESP32. Esta función es vital para cumplir con los objetivos de ”mecanismos de diagnóstico, reconexión y trazabilidad”definidos en el plan de pasantía.

Dificultades

Implementando una nueva versión que sea escalable entramos en la problematica de ”demasiado uso de memoria”:

Memory Type Usage Summary

Memory Type/Section	Used [bytes]	Used [%]	Remain [bytes]	Total [bytes]
Flash Code	77774			
.text	77774			
IRAM	57659	43.99	73413	131072
.text	56631	43.21		
.vectors	1028	0.78		
Flash Data	40140			
.rodata	39884			
.appdesc	256			
DRAM	11312	6.26	169424	180736
.data	8896	4.92		
.bss	2416	1.34		
RTC SLOW	56	0.68	8136	8192
.force_slow	32	0.39		
.rtc_slow_reserved	24	0.29		

Total image size: 184501 bytes (.bin may be padded larger)
Note: The reported total sizes may be smaller than those in the technical reference manual due to reserved memory and application configuration. The total flash size available for the application is not included by default, as it cannot be reliably determined due to the presence of other data like the bootloader, partition table, and application partition size.

Figura 5: Uso excesivo de memoria IRAM (43.99 %).

Siendo esta la implementación de una unica tarea, por lo tanto se toma como una actualización pero se buscara realizar una version optimizada de la misma.

Notas

Vemos un vistazo de como serán los tópicos enviados a través de MQTT, culminando una primera versión para la captura y procesamiento de datos:

```
I (35473) uart_events: [MQTT SIM] Datos listos para enviar:
I (35473) uart_events: [MQTT SIM] Datos listos para enviar:
I (35473) uart_events: [MQTT SIM] Serial: Z1F9999988
I (35473) uart_events: [MQTT SIM] Serial: Z1F9999988
I (35473) uart_events: [MQTT SIM] STS1: 0x60, STS2: 0x40
I (35473) uart_events: [MQTT SIM] Estado: Modo Fiscal y en Espera
I (35473) uart_events: [MQTT SIM] Publicar: v1/fiscal/Z1F9999988/sts1 -> 0x60
I (35483) uart_events: [MQTT SIM] Publicar: v1/fiscal/Z1F9999988/sts2 -> 0x40
I (35493) uart_events: [MQTT SIM] Publicar: v1/fiscal/Z1F9999988/estado -> Modo Fiscal y en Espera
I (35503) uart_events: [MQTT SIM] Publicar: v1/fiscal/Z1F9999988/error -> Ningún error
I (35503) uart_events: [MQTT SIM] Publicar: v1/fiscal/Z1F9999988/serial -> Z1F9999988
I (35513) uart_events: [MQTT SIM] Publicar: v1/fiscal/Z1F9999988/timestamp -> 33470
I (35523) uart_events: [MQTT SIM] Intervalo desde último estado: 5060 ms
```

Figura 6: Tópicos para protocolo MQTT y datos contenidos en el tópico.

Con estos datos ya parametrizados, el código esta casi listo para comenzar la implementación WiFi y MQTT, no obstante, en estos momentos centrare el trabajo en resolver la alta demanda de memoria que actualmente tiene el código. Una vez resuelto, si avanzaremos con la implementación WiFi y MQTT.

Fecha: 03/02/2026

Actividades

- Realizar conexión WiFi para Firmware V1.
- Realizar conexión MQTT para Firmware V1.
- Visualizar primeros datos a través de broker HiveMQ en la nube.

Aprendizajes

Usando programación modular para gestionar las extensas tareas del firmware, se propone la siguiente estructura:

```
firmware/
├── main/
|   ├── ismartv1.c          # app_main y configuración general
|   ├── printer_config.h    # Configuraciones, constantes, estructuras
|   ├── printer_task.h      # Declaraciones de funciones de la tarea de impresora
|   ├── printer_task.c      # Implementación de la tarea de impresora (con reconexión)
|   ├── printer_mqtt.h      # Declaraciones para MQTT (sin conectar aún)
|   └── printer_mqtt.c      # Implementación de MQTT (simulador por ahora)
```

Figura 7: Estructura modular para Firmware V1.

Dicha estructura mostro un aumento de 0.11 % en el uso de la memoria IRAM, pero en investigaciones pude encontrar que, un uso del 44.01 % de la IRAM (Instruction RAM) en un proyecto basado en el ESP32 utilizando el framework ESP-IDF no se considera excesivo, pero sí es un valor que requiere atención y monitoreo constante a medida que se integren más funcionalidades.

Memory Type Usage Summary

Memory Type/Section	Used [bytes]	Used [%]	Remain [bytes]	Total [bytes]
Flash Code	80358			
.text	80358			
IRAM	57683	44.01	73389	131072
.text	56655	43.22		
.vectors	1028	0.78		
Flash Data	43100			
.rodata	42844			
.appdesc	256			
DRAM	11276	6.24	169460	180736
.data	8932	4.94		
.bss	2344	1.3		
RTC SLOW	56	0.68	8136	8192
.force_slow	32	0.39		
.rtc_slow_reserved	24	0.29		

Total image size: 190105 bytes (.bin may be padded larger)
Note: The reported total sizes may be smaller than those in the technical reference manual due to reserved memory and application configuration. The total flash size available for the application is not incl
uded by default, as it cannot be reliably determined due to the presence of other data like the bootloader, partition table, and application partition size.

Figura 8: Tabla del uso de memoria para programación modular.

Se adjuntan ejemplos de uso:

```
I (164306) printer_events: ENQ enviado, esperando respuesta...
I (164346) printer_events: Bytes recibidos: 4
I (164346) printer_events: Trama válida recibida
I (164346) printer_events: STX: 0x02, STS1: 0x60, STS2: 0x40, ETX: 0x03
I (164346) printer_events: Modo Fiscal y en Espera
I (164346) printer_events: Ningún error
I (164356) mqtt_events: === DATOS PREPARADOS PARA MQTT ===
I (164356) mqtt_events: Serial: Z1F9999988
I (164366) mqtt_events: STS1: 0x60, STS2: 0x40
I (164366) mqtt_events: Estado: Modo Fiscal y en Espera
I (164376) mqtt_events: Error: Ningún error
I (164376) mqtt_events: ================================
I (164376) printer_events: Impresora en modo fiscal, solicitando STATUS S1...
I (164486) printer_events: STATUS S1 solicitado, esperando...
I (164646) printer_events: STATUS S1 recibido: 116 bytes
I (164646) mqtt_events: Serial actualizado: Z1F9999988
I (164646) printer_events: STATUS S1 procesado exitosamente
I (164646) printer_events: Serial: Z1F9999988
I (164646) printer_events: Esperando 30000 ms antes del próximo ciclo...
I (165806) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/sts1 -> 0x60
I (165806) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/sts2 -> 0x40
I (165806) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/estado -> Modo Fiscal y en Espera
I (165816) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/error -> Ningún error
I (165816) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/cajero -> 0010
I (165826) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/ventas -> 0000000000000000
I (165836) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/ultima_factura -> 00000000
I (165846) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/facturas_emitidas -> 00000
I (165846) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/ultima_nota_debito -> 00000000
I (165856) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/cantidad_notas_debito -> 00000
I (165866) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/ultima_nota_credito -> 00000000
I (165876) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/cantidad_notas_credito -> 00000
I (165886) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/ultimo_doc_nofiscal -> 00000000
I (165886) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/cantidad_docs_nofiscal -> 00000
I (165896) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/cierres_z -> 0000
I (165906) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/reportes_fiscal -> 0000
I (165916) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/rif -> J3121711970
I (165916) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/hora -> 123000
I (165926) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/fecha -> 300126
I (165936) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/timestamp -> 164370
I (165936) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/errores -> 0
I (165946) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/reconexiones -> 0
I (165956) mqtt_events: [SIM] Publicar: v1/fiscal/Z1F9999988/json_completo -> {"timestamp":164370,"sts1":"0x60","sts2":"0x40","estado":"Modo Fiscal y en Espe
ra","error":"Ningún error","cajero":"0010","ventas":"0000000000000000","ult_factura":"00000000","fact_emitidas":"00000","ult_debito":"00000000","cant_debito
":"00000","ult_credito":"00000000","cant_credito":"00000","ult_nofiscal":"00000000","cant_nofiscal":"00000","cierres_z":"0000","reportes_fiscal":"0000","rif"
:"J3121711970","registro":"Z1F9999988","hora":"123000","fecha":"300126"}
I (165996) mqtt_events: Intervalo desde última sincronización: 130200 ms
```

Figura 9: Muestra de procesamiento de datos capturados y preparados para enviar por MQTT.

```
I (62196) printer_events: ENQ enviado, esperando respuesta...
W (64196) printer_events: No se recibió respuesta (timeout)
E (64196) printer_events: Error 1 - Total errores: 1
I (64196) printer_events: Esperando 30000 ms antes del próximo ciclo...
I (87426) MAIN: Ciclo 30 - Estado impresora: WAITING_RESPONSE
I (87426) MAIN: Memoria libre: 293684 bytes
W (94196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
I (95196) printer_events: Ciclo 5 - Estado: WAITING_RESPONSE, Errores: 1
W (95196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (96196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (97196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (98196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (99196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
I (100196) printer_events: Ciclo 10 - Estado: WAITING_RESPONSE, Errores: 1
W (100196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (101196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (102196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (103196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (104196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
I (105196) printer_events: Ciclo 15 - Estado: WAITING_RESPONSE, Errores: 1
W (105196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (106196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (107196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (108196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (109196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
I (110196) printer_events: Ciclo 20 - Estado: WAITING_RESPONSE, Errores: 1
W (110196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (111196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (112196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (113196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (114196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
I (115196) printer_events: Ciclo 25 - Estado: WAITING_RESPONSE, Errores: 1
W (115196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (116196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (117196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
I (117426) MAIN: Ciclo 40 - Estado impresora: WAITING_RESPONSE
I (117426) MAIN: Memoria libre: 293684 bytes
W (118196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (119196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
I (120196) printer_events: Ciclo 30 - Estado: WAITING_RESPONSE, Errores: 1
W (120196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (121196) printer_events: Esperando estado IDLE. Estado actual: WAITING_RESPONSE
W (122196) printer_events: Sin respuesta por mucho tiempo, considerando reconexión
W (122196) printer_events: Necesita reconexión. Estado actual: WAITING_RESPONSE
W (122196) printer_events: Intentando reconexión 1/5...
I (122196) printer_events: UART desinicializado
```

Figura 10: Muestra de funcionalidad de reconexión.

Dificultades

Notas

Ahora avanzaremos a la siguiente fase del proyecto Ïmplementación de comunicación Wi-Fi y MQTT en el ESP32"para ello se implementará una adaptación del mini-proyecto realizado en semana 1, a lo que tenemos de simulador MQTT en la versión actual.

```
I (153435) printer_events: STX: 0x02, STS1: 0x60, STS2: 0x40, ETX: 0x03
I (153435) printer_events: Modo Fiscal y en Espera
I (153435) printer_events: Ningún error
I (153445) mqtt_events: === DATOS PREPARADOS PARA MQTT ===
I (153445) mqtt_events: Serial: Z1F9999988
I (153455) mqtt_events: STS1: 0x60, STS2: 0x40
I (153455) mqtt_events: Estado: Modo Fiscal y en Espera
I (153465) mqtt_events: Error: Ningún error
I (153465) mqtt_events: ==================================
I (153465) printer_events: Impresora en modo fiscal, solicitando STATUS S1...
I (153575) printer_events: STATUS S1 solicitado, esperando...
I (153725) printer_events: STATUS S1 recibido: 116 bytes
I (153725) mqtt_events: Serial actualizado: Z1F9999988
I (153725) printer_events: STATUS S1 procesado exitosamente
I (153725) printer_events: Serial: Z1F9999988
I (153725) printer_events: Esperando 30000 ms antes del próximo ciclo...
I (156475) mqtt_events: Sincronizando datos (5)...
E (156475) mqtt_events: Publicado: v1/fiscal/Z1F9999988/json_completo -> {"timestamp":153170,"sts1":"0x60","sts2":"0x40",
estado":"Modo Fiscal y en Espera","error":"Ningún error","cajero":"0010","ventas":"00000000000000000","ult_factura":"0000
000","fact_emitidas":"00000","ult_debito":"00000000","cant_debito":"00000","ult_credito":"00000000","cant_credito":"00000
,"ult_nofiscal":"00000000","cant_nofiscal":"00000","cierres_z":"0000","reportes_fiscal":"0000","rif":"J3121711970","regis
ro":"Z1F9999988","hora":"123000","fecha":"300126"}
I (156515) mqtt_events: Intervalo desde última sincronización: 30050 ms
I (156525) mqtt_events: Sincronización completada (5)
I (156735) mqtt_events: MQTT publicado, msg_id=64275
```

Figura 11: Prueba de funcionamiento para datos enviados desde MQTT.

Figura 12: Visualización de datos a través de broker HiveMQ.

Se logró enviar los datos a través de MQTT hacia el broker escogido. Además se implemento un sistema de reconexión en caso de si se va el wifi o se pierde conexion con la impresora. Se concluye que los objetivos del dia fueron cumplidos.

Fecha: 04/02/2026

Actividades

- Crear casos para estados $\neq$ 0x60 donde no se envien todos los datos (esto por que en una impresora real ocurre así).
- Implementar LCR del protocolo ENQ y STATUS S1.
- Arreglar definiciones redundantes en el codigo.
- Mejorar el manejo de IRAM para futuras actualizaciones (seguridad TLS y OTA).

Aprendizajes

Dificultades

No se reportan muchos avances, ya que se sigue trabajando en la nueva adaptación que trabaje lo mas fiel posible al protocolo de comunicación de una impresora fiscal.

Notas

Aplique mejoras al simulador (`impresora.py`) coon el fin de que sea lo mas cercano a una impresora fiscal, y el código de firmware sea lo mas fiel a futuras integraciones fisicas reales. Ahora nuestro simulador maneja LCR, asi como tambien el envio de ACK y NAK.

Fecha: 06/02/2026

Actividades

- Corregir incovenientes con el código.

Aprendizajes

¿Qué es un Mutex?

Imagina que `mqtt_data` es un cuaderno donde escribes información.

- La **Tarea de la Impresora** es alguien que escribe en el cuaderno.
- La **Tarea de MQTT** es alguien que lee del cuaderno para enviar la información a internet.

Si ambos intentan usar el cuaderno al mismo tiempo, uno podría estar leyendo mientras el otro apenas va por la mitad de la escritura, y los datos se enviarían incompletos o corruptos.

Un Mutex (abreviatura de Mutual Exclusion) es como una llave.

- Para tocar el cuaderno, tienes que pedir la llave.
- Si la tienes tú, nadie más puede tocarlo.
- Cuando terminas, devuelves la llave para que otro la use.

Dificultades

Memory Type Usage Summary

Memory Type/Section	Used [bytes]	Used [%]	Remain [bytes]	Total [bytes]
Flash Code	651014			
.text	651014			
Flash Data	199836			
.rodata	199580			
.appdesc	256			
IRAM	102751	78.39	28321	131072
.text	101723	77.61		
.vectors	1028	0.78		
DRAM	37852	20.94	142884	180736
.bss	21688	12.0		
.data	16164	8.94		
RTC SLOW	56	0.68	8136	8192
.force_slow	32	0.39		
.rtc_slow_reserved	24	0.29		

Figura 13: Uso de IRAM con implementación completa muy elevado (78.39 %).

Notas

Se lograron significativos avances en la mejora de IRAM para la tarea '`printer_task.c`' y se buscará seguir avanzando con la misma metodologia con las demas tareas.

Fecha: 09/02/2026

Actividades

- Mejorar implementación para `mqtt_task.c`

Aprendizajes

Configurando `menuconfig` logramos una mejora con el uso de la IRAM desactivando la optimizacion por IRAM de la conexion WiFi.

Memory Type Usage Summary

Memory Type/Section	Used [bytes]	Used [%]	Remain [bytes]	Total [bytes]
Flash Code	613368			
.text	613368			
Flash Data	190252			
.rodata	189996			
.appdesc	256			
IRAM	68495	52.26	62577	131072
.text	67467	51.47		
.vectors	1028	0.78		
DRAM	37424	20.71	143312	180736
.bss	21656	11.98		
.data	15768	8.72		
RTC SLOW	56	0.68	8136	8192
.force_slow	32	0.39		
.rtc_slow_reserved	24	0.29		

Figura 14: Reducción del uso de la IRAM en un -26.13 % (ahora 52.26 %).

Dificultades

Con la implementación de ACK y NAK el UART genera mucha basura y no se logra una buena comunicación entre el simulador de impresora y el ESP32, se trabaja para solucionarlo.

Notas

Fecha: 10/02/2026

Actividades

- Solucionar problemas con la comunicación UART.

Aprendizajes

A veces, cuando el edificio se está cayendo, lo mejor es salir, construir una pequeña cabaña al lado y verificar que los cimientos funcionan. Por ello se buscara hacer una prueba del funcionamiento para UART0 y UART1, realizando un proyecto pequeño donde se comuniquen entre si, buscando recepción y transmision optima, libre de ruido y completamente funcional.

0.0.1. User Properties de MQTTv5

Las User Properties son, básicamente, metadatos en formato clave-valor (UTF-8) que se puede añadir a las cabeceras de casi cualquier paquete MQTT (CONNECT, PUBLISH, SUBSCRIBE, etc.). Las User Properties son como etiquetas adhesivas que pegas por fuera de la caja. Se pueden leer sin abrir el paquete. Esto nos sirve para la implementación web donde no será necesario ver el contenido del mensaje MQTT para direccionarlo a través del backend.

Dificultades

A pesar de haber aplicado un proyecto minimo (y este haya funcionado) nuestro código sigue sin poner leer el estado enviado desde nuestro simulador:

```
I (64467) printer_events: Configurando UART...
I (64467) printer_events: Instalando driver UART...
I (64467) uart: queue free spaces: 40
I (64467) printer_events: Driver UART instalado
I (64467) printer_events: Configurando parámetros UART...
I (64477) printer_events: Parámetros UART configurados
I (64477) printer_events: Configurando pines UART: TX=4, RX=5
I (64487) printer_events: Pines UART configurados
I (64487) printer_events: UART completamente inicializado y listo
I (64497) printer_events: Buffer: 2048 bytes, Cola: 40
I (64497) printer_events: UART inicializado y limpio
I (64507) printer_events: Reconexión exitosa
I (64507) printer_events: Enviando comando ENQ...
I (65137) printer_events: Sin respuesta al ENQ (Timeout)
E (65137) printer_events: Error 1 - Total errores: 1
```

```
¡ENQ RECIBIDO! El ESP32 nos busca.
→STATUS enviado (LRC: 23)
(Byte: 00)

¡ENQ RECIBIDO! El ESP32 nos busca.
→STATUS enviado (LRC: 23)
(Byte: 00)

¡ENQ RECIBIDO! El ESP32 nos busca.
→STATUS enviado (LRC: 23)
(Byte: 00)

¡ENQ RECIBIDO! El ESP32 nos busca.
→STATUS enviado (LRC: 23)
```

Figura 15: ESP32 no lee STATUS enviado por simulador.

Notas

Con los problemas solucionados se logro una sincronizacion y optimizacion casi perfectas de esta primera versión de firmware enviando alrededor de mas de 160 mensaje a través de protocolo MQTT sin gran complicación para el firmware.

```
I (104342) printer_events: Trama completa recibida (5 bytes)
I (104342) printer_events: Enviando comando S1...
I (104442) printer_events: Comando S1 enviado, esperando ACK...
I (104502) printer_events: ACK/NAK recibido: 0x06
I (104502) printer_events: ACK recibido, esperando STATUS S1...
I (104852) printer_events: Trama completa recibida (117 bytes)
I (104852) printer_events: STATUS S1 válido → ACK enviado
I (104852) mqtt_events: Serial actualizado: Z1F9999988
I (104852) printer_events: Datos de STATUS S1 actualizados
I (104852) mqtt_events: Sincronizando datos (19)...
I (104862) printer_events: Ciclo completado exitosamente
I (104872) mqtt_events: Mensaje enviado con éxito [ID: 43906]
I (104872) mqtt_events: Sincronización completada (19)
I (109872) printer_events: Enviando comando ENQ...
I (109882) printer_events: Trama completa recibida (5 bytes)
I (109882) printer_events: Enviando comando S1...
I (109982) printer_events: Comando S1 enviado, esperando ACK...
I (109992) printer_events: ACK/NAK recibido: 0x06
I (109992) printer_events: ACK recibido, esperando STATUS S1...
I (110332) printer_events: Trama completa recibida (117 bytes)
I (110332) printer_events: STATUS S1 válido → ACK enviado
I (110332) mqtt_events: Serial actualizado: Z1F9999988
I (110332) printer_events: Datos de STATUS S1 actualizados
I (110332) mqtt_events: Sincronizando datos (20)...
I (110342) printer_events: Ciclo completado exitosamente
I (110352) mqtt_events: Mensaje enviado con éxito [ID: 22799]
I (110352) mqtt_events: Sincronización completada (20)
```

```
→Respuesta S1 enviada (117 bytes)
.→STATUS enviado (LRC: 23)

- STX recibido, capturando trama...
✅ Trama RX: 02 53 31 03 61
→ACK enviado (0x06)
Comando: S1 (5331)
↳ Solicitud de Reporte S1 detectada
→Respuesta S1 enviada (117 bytes)
.→STATUS enviado (LRC: 23)

- STX recibido, capturando trama...
✅ Trama RX: 02 53 31 03 61
→ACK enviado (0x06)
Comando: S1 (5331)
↳ Solicitud de Reporte S1 detectada
→Respuesta S1 enviada (117 bytes)
.→STATUS enviado (LRC: 23)

- STX recibido, capturando trama...
✅ Trama RX: 02 53 31 03 61
→ACK enviado (0x06)
Comando: S1 (5331)
↳ Solicitud de Reporte S1 detectada
→Respuesta S1 enviada (117 bytes)
```

Figura 16: Firmware funcionando correctamente con protocolo MQTT.

Fecha: 11/02/2026

Actividades

- Implementar USER PROPERTIES.
- Idear protocolo para username y pasword.

Aprendizajes

Dificultades

Notas

Fecha: XX/XX/2026

Actividades

Aprendizajes

Dificultades

Notas